List Scheduling of Dependent Non-Preemptive Tasks: Priority Rules, Guarantees and an Empirical Comparison

Olivier P.

Abstract

We study the minimisation of the makespan of a set of tasks with precedence constraints on m identical machines without preemption, $P \,|\, \mathrm{prec} \,|\, C_{\max}$. The problem is strongly NP-hard, so we consider list scheduling, for which Graham's bound guarantees a ratio $2 - 1/m$ whatever the priority rule. We define the classical priorities (height, depth, t-level, b-level, number of successors, duration), a fuzzy tie-breaking rule, and a genetic algorithm that searches over convex combinations of these priorities. On 180 random graphs of 1000 tasks, the b-level rule shortens the makespan by 2.43% on average relative to the height rule (standard error 0.30); the fuzzy tie-break and the genetic algorithm give no significant further gain.

1. Problem

An instance is a directed acyclic graph $G = (V, E)$, $|V| = n$, with durations $p_j > 0$, and m identical machines. A schedule assigns start times $S_j \geq 0$ and machines such that (i) $S_j \geq S_i + p_i$ for all $(i, j) \in E$; (ii) a task runs uninterrupted on one machine during $[S_j, S_j + p_j)$; (iii) at most m tasks run at any time. The objective is $C_{\max} = \max_j (S_j + p_j)$. In the notation of Graham et al. [4] this is $P \,|\, \mathrm{prec} \,|\, C_{\max}$; it is NP-hard already for unit durations [10] and strongly NP-hard in general [8], which rules out exact methods on graphs with thousands of tasks; see [7] for a survey of heuristics.

Lower bound. Let the length of a path be the sum of its durations and CP the length of a longest (critical) path. Any schedule satisfies

$$C^*_{\max} \geq \mathrm{LB} = \max\Big(\mathrm{CP},\ \tfrac{1}{m}\textstyle\sum_j p_j\Big), \tag{1}$$

since the tasks of a path run sequentially and the total work is shared by m machines.

2. List scheduling and Graham's bound

Given a priority $\pi : V \to \mathbb{R}$, list scheduling is the event-driven rule: whenever a machine is idle and the set of *ready* tasks (all predecessors finished) is non-empty, start the ready task of highest priority.

Theorem 1 (Graham [2, 3])**.** *For every priority π, the list schedule satisfies* $C_{\max} \leq \frac{1}{m}\sum_j p_j + \big(1 - \frac{1}{m}\big)\mathrm{CP} \leq \big(2 - \frac{1}{m}\big) C^*_{\max}$.

Proof. Let j_k be a task finishing last and $C_j = S_j + p_j$. Given j_ℓ, let $j_{\ell-1}$ be a predecessor of j_ℓ finishing last. On $[C_{j_{\ell-1}}, S_{j_\ell})$, j_ℓ is ready, so no machine is idle there (the rule never idles a machine while a task is ready). Stop at a task j_1 without predecessors, which is ready on $[0, S_{j_1})$. Hence at every instant of $[0, C_{\max})$ not covered by a chain task, all m machines are busy. Counting busy time, $mC_{\max} \leq \sum_j p_j + (m-1)\sum_\ell p_{j_\ell} \leq \sum_j p_j + (m-1)\mathrm{CP}$. Conclude with (1). □

The bound is tight in the worst case and list scheduling exhibits anomalies: adding a machine or shortening a task may increase $C_{\max}$ [3]. The choice of π therefore matters only on average, which is what we measure.

3. Priority rules

With $\mathrm{pred}(j)$, $\mathrm{succ}(j)$ the immediate neighbours, the rules are computed by one pass in topological (resp. reverse topological) order, in $O(n + |E|)$:

$$\begin{aligned} \mathrm{height}(j) &= \max\nolimits_{i \in \mathrm{pred}(j)} \mathrm{height}(i) + 1, \\ \mathrm{depth}(j) &= \max\nolimits_{k \in \mathrm{succ}(j)} \mathrm{depth}(k) + 1, \\ t\ell(j) &= p_j + \max\nolimits_{i \in \mathrm{pred}(j)} t\ell(i), \\ b\ell(j) &= p_j + \max\nolimits_{k \in \mathrm{succ}(j)} b\ell(k), \end{aligned}$$

with $\max \emptyset = 0$; to these we add $|\mathrm{succ}(j)|$ and p_j. The b-level is the length of the longest path starting at j: even with infinitely many machines, nothing can finish before $S_j + b\ell(j)$, so $C_{\max} \geq \max_j (S_j + b\ell(j))$ and the task with the largest b-level is the one whose delay is most likely to delay the whole schedule; $\max_j b\ell(j) = \mathrm{CP}$. Depth is the b-level with unit durations. For in-trees with unit durations, the highest-level-first rule is optimal [6], and Coffman–Graham is optimal for $m = 2$ [1]; b-level is the natural extension to arbitrary durations and is the core of HEFT [9].

Fuzzy tie-breaking. Among tasks of equal b-level, a Mamdani-type rule base [11] on height and duration, with triangular memberships $\mu(x; a, b, c) = \max\big(0, \min(\frac{x-a}{b-a}, \frac{c-x}{c-b})\big)$, the min t-norm and centroid defuzzification, gives a secondary key. It can only permute tasks inside classes of equal b-level, which are small when durations are continuous; its effect is therefore expected to be small.

4. Genetic search over priorities

Let $f_1, \ldots, f_6$ be the six features above, normalised to $[0, 1]$. A chromosome encodes $w \in \{0, \ldots, 255\}^6$ (48 bits) and the priority $\pi_w(j) = \sum_k w_k f_k(j)$; its fitness is the makespan of the corresponding list schedule. Selection keeps the best chromosomes, single-point crossover (rate 0.5) and bit-flip mutation (rate 0.2) produce offspring [5]. Every pure rule is a point of the search space ($w = e_k$), so if the population is seeded with the six pure rules and the best chromosome is always kept (elitism), the best makespan found is non-increasing and never worse than the best pure rule; without this seeding, nothing guarantees it.

5. Experiments

Graphs are random DAGs, m is drawn between 2% and 5% of n, and performance is $R_A = 100\,(1 - C^A_{\max}/C^{\mathrm{height}}_{\max})$ (positive is better). Table 1 reports mean and standard deviation over 180 graphs, with standard error $s/\sqrt{180}$.

Findings. (1) b-level dominates: its mean gain is about 8 standard errors above zero, and depth, its unit-

	$n = 1000$		$n = 50$
Rule	mean (sd)	s.e.	mean (sd)
b-level	2.43 (4.01)	0.30	3.11 (3.39)
b-level + fuzzy	2.43 (4.00)	0.30	3.13 (3.40)
depth	1.79 (3.05)	0.23	1.11 (2.77)
successors	−0.14 (1.75)	0.13	0.27 (2.41)
duration	−0.29 (1.35)	0.10	0.61 (3.34)
t-level	−1.08 (2.18)	0.16	−2.34 (4.00)
genetic	–	–	0.25 (2.28)

Table 1: Improvement over the height rule (%).

duration version, is second, which confirms that the information carried by the remaining critical path is what matters. (2) t-level is worse than the baseline: it favours tasks far from the sources, i.e. it looks backwards when the makespan is determined by what remains. (3) The fuzzy tie-break does not differ from b-level within noise, as predicted. (4) The genetic algorithm, with population 30 and 20 generations (600 fitness evaluations), did not beat b-level on $n = 50$ and was by far the slowest; on such small graphs most priorities give the same schedule, so selection has no signal. All rules except the genetic algorithm run within 0.1 s of each other.

Conclusion. b-level list scheduling is, at equal cost, the best of the tested rules, with Graham's $2 - 1/m$ guarantee. A seeded, elitist genetic search, or a hybrid with local search, is the natural next step to obtain further improvement on large graphs.

References

[1] E. Coffman, R. Graham. Optimal scheduling for two-processor systems. *Acta Informatica* 1:200–213, 1972.

[2] R. Graham. Bounds for certain multiprocessing anomalies. *Bell Syst. Tech. J.* 45(9):1563–1581, 1966.

[3] R. Graham. Bounds on multiprocessing timing anomalies. *SIAM J. Appl. Math.* 17(2):416–429, 1969.

[4] R. Graham, E. Lawler, J. Lenstra, A. Rinnooy Kan. Optimization and approximation in deterministic sequencing and scheduling: a survey. *Ann. Discrete Math.* 5:287–326, 1979.

[5] J. Holland. *Adaptation in Natural and Artificial Systems*. Univ. of Michigan Press, 1975.

[6] T. C. Hu. Parallel sequencing and assembly line problems. *Operations Research* 9(6):841–848, 1961.

[7] Y.-K. Kwok, I. Ahmad. Static scheduling algorithms for allocating directed task graphs to multiprocessors. *ACM Comput. Surv.* 31(4):406–471, 1999.

[8] J. Lenstra, A. Rinnooy Kan. Complexity of scheduling under precedence constraints. *Operations Research* 26(1):22–35, 1978.

[9] H. Topcuoglu, S. Hariri, M.-Y. Wu. Performance-effective and low-complexity task scheduling for heterogeneous computing. *IEEE TPDS* 13(3):260–274, 2002.

[10] J. Ullman. NP-complete scheduling problems. *J. Comput. Syst. Sci.* 10(3):384–393, 1975.

[11] L. Zadeh. Fuzzy sets. *Information and Control* 8(3):338–353, 1965.